```
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

data = load_breast_cancer()
X, y = data.data, data.target

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClas
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(max_depth=5),
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "SVM": SVC(),
    "Naive Bayes": GaussianNB(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='loglos
}
```

```
results = {}

for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    results[name] = scores.mean()

results_df = pd.DataFrame(list(results.items()), columns=["Model", "Accuracy
results_df
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

	Model	Accuracy
0	Logistic Regression	0.980686
4	SVM	0.973638
7	XGBoost	0.973638
2	Random Forest	0.964881
3	KNN	0.964850
6	Gradient Boosting	0.963127
5	Naive Bayes	0.927930
1	Decision Tree	0.922652

Next steps: Generate code with `results_df` New interactive sheet

```
from sklearn.datasets import fetch_20newsgroups

categories = ['alt.atheism', 'comp.graphics', 'sci.med', 'rec.sport.baseball

train_data = fetch_20newsgroups(subset='train', categories=categories, remov
test_data = fetch_20newsgroups(subset='test', categories=categories, remove=

X_train, y_train = train_data.data, train_data.target
X_test, y_test = test_data.data, test_data.target
```

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

svm_pipeline = Pipeline([
```

```
    ('tfidf', TfidfVectorizer(stop_words='english', max_df=0.7)),
    ('svm', LinearSVC())
])

svm_pipeline.fit(X_train, y_train)
```

```
▸ Pipeline
  ▸ TfidfVectorizer
  ▸ LinearSVC
```

```
from sklearn.metrics import accuracy_score

svm_preds = svm_pipeline.predict(X_test)
svm_acc = accuracy_score(y_test, svm_preds)
svm_acc
```

```
0.882744836775483
```

```
from sklearn.linear_model import LogisticRegression

lr_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', max_df=0.7)),
    ('lr', LogisticRegression(max_iter=1000))
])

lr_pipeline.fit(X_train, y_train)

lr_preds = lr_pipeline.predict(X_test)
lr_acc = accuracy_score(y_test, lr_preds)
lr_acc
```

```
0.8807461692205196
```

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, StratifiedKFold, cross
from sklearn.preprocessing import StandardScaler
from scipy.stats import ttest_rel
```

```
def model_selection_report(X, y, models_dict):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
```

```
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    results = {}

    for name, model in models_dict.items():
        scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='ac
        results[name] = scores

    df = pd.DataFrame(results)

    mean_scores = df.mean().sort_values(ascending=False)
    best_model = mean_scores.index[0]

    stats = {}
    for model in df.columns:
        if model != best_model:
            stat, p = ttest_rel(df[best_model], df[model])
            stats[model] = p

    return df, best_model, stats
```

```
df, best_model, stats = model_selection_report(X, y, models)
df, best_model, stats
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
(   Logistic Regression  Decision Tree  Random Forest       KNN       SVM  \
 0            0.967033       0.945055       0.956044  0.956044  0.945055
 1            0.989011       0.934066       0.956044  0.967033  0.978022
 2            0.978022       0.912088       0.934066  0.945055  0.956044
 3            0.989011       0.923077       0.956044  0.956044  0.967033
 4            0.967033       0.901099       0.989011  0.978022  0.989011

    Naive Bayes  Gradient Boosting   XGBoost
 0     0.912088           0.967033  0.978022
 1     0.967033           0.956044  0.978022
 2     0.890110           0.934066  0.945055
 3     0.956044           0.945055  0.956044
```

```
 4     0.945055           0.978022  0.967033  ,
 'Logistic Regression',
 {'Decision Tree': np.float64(0.0029654840709450116),
  'Random Forest': np.float64(0.1671033157325984),
  'KNN': np.float64(0.09930068321372658),
  'SVM': np.float64(0.26626462796630995),
  'Naive Bayes': np.float64(0.024711379456504583),
  'Gradient Boosting': np.float64(0.12920298463940977),
  'XGBoost': np.float64(0.2079999999999989)})
```

```
def algorithm_selector(n_samples, n_features, is_linear, data_type, need_int

    if data_type == "text":
        return "TF-IDF + Linear SVM or Logistic Regression"

    if n_samples < 1000:
        if n_features > n_samples:
            return "Linear SVM or Logistic Regression"
        else:
            return "Naive Bayes or Logistic Regression"

    if is_linear:
        return "Logistic Regression or Linear SVM"

    if need_interpretability:
        return "Decision Tree or Logistic Regression"

    if n_samples > 100000:
        return "XGBoost or SGDClassifier"

    return "Random Forest or Gradient Boosting"
```

```
algorithm_selector(500, 100, True, "tabular", False)
```

```
'Naive Bayes or Logistic Regression'
```